



COMP 4300

Intro to Game Programming

Lecture 3

SFML + ImGui Tutorial

SFML

- Simple and Fast Multimedia Library
- Multi-Platform, [Multi-Language](#)
- Handles the things we don't want to
 - Windows, Rendering, Audio, Images, etc
- Based on OpenGL
 - Can be used with raw [OpenGL code](#)
- Super easy to [learn and use](#)

SFML Tutorial

ImGui

- Immediate Mode GUI
- Amazing for dev/debugging GUI
 - Can customize, but not super pretty
- Works by **directly modifying variables** in memory rather than callback functions
- Works very well with Game Loop structure
- Easy to learn and implement

Game Loops

```
    // display initial GUI screen
1. while (true)
2. {
3.     // process user input if any exists
4.     // update internal game simulation
5.     // render current game screen
6. }
```

Game Loops

```
// display initial GUI screen
1. while (true)
2. {
3.     processInput();
4.     update();
5.     render();
6. }
```

Widgets

```
ImGui::Checkbox("Draw Shape", &shape.draw);
```

```
ImGui::SliderInt("SomeInt", &intVariable, min, max);
```

```
ImGui::SliderFloat("Scale", &shape.scale, min, max);
```

```
float v[2] = { 0.0, 0.0 };
```

```
ImGui::SliderFloat2("Velocity", v, min, max);
```

```
float shapeColor[3] = {0.0f, 0.0f, 0.0f};
```

```
ImGui::ColorEdit3("Color", shapeColor);
```

Widgets

```
char buffer[255];  
ImGui::InputText("Default", buffer, 255);  
  
if (ImGui::Button("Button Text"))  
{  
    // this happens when button is pressed  
}  
ImGui::SameLine();  
if (ImGui::Button("Button Two"))  
{  
    // this will be on the same line  
}
```


Tabs

```
if (ImGui::BeginTabBar("MyTabBar"))
{
    if (ImGui::BeginTabItem("Avocado"))
    {
        ImGui::Text("This is the Avocado tab!");
        ImGui::EndTabItem();
    }
    if (ImGui::BeginTabItem("Broccoli"))
    {
        ImGui::Text("This is the Broccoli tab!");
        ImGui::EndTabItem();
    }
    ImGui::EndTabBar();
}
```

Combo Box

// by default, combo uses raw string arrays

```
const char* items[] = { "AAAA", "BBBB", "CCCC", "DDDD" };  
static int item_index = 0;  
ImGui::Combo("Combo", &item_index, items, IM_ARRAYSIZE(items));
```

// you may find it easier to work with vectors

```
std::vector<const char*> items;  
for (auto& s : shapes) { items.push_back(s.nameString.c_str()); }  
int item_index = 0;  
ImGui::Combo("Shape", &item_index, items.data(), (int)items.size());
```

ImGui Naming

```
// ImGui elements need to be given unique names, or the last  
// one will override the behavior of any other of the same name
```

```
if (ImGui::Button("Delete")) { deleteThingOne(); }  
if (ImGui::Button("Delete")) { deleteThingTwo(); }
```

```
// Can fix this by adding ## to name, which will not display
```

```
if (ImGui::Button("Delete##D1")) { deleteThingOne(); }  
if (ImGui::Button("Delete##D2")) { deleteThingTwo(); }
```

ImGui Demo / Examples